Mục lục

6 Vi điều khiển và nền tảng phần cứng IoT **1**

6.1 Kiến trúc hệ thống nhúng và mục tiêu học tập 1

6.2 ATmega328P: nền tảng AVR 8 bit 2

6.2.1 Flash, SRAM, EEPROM: ba vai trò khác nhau 2

6.2.2 Chân nguồn, I/O và nạp chương trình 2

6.3 Arduino: hệ sinh thái phần cứng/phần mềm mở 2

6.3.1 Các bo Arduino trong bài giảng 2

6.3.2 Phân tích một bo Uno cổ điển 3

6.3.3 Shield, breadboard, kit và các nền tảng thay thế 3

6.4 Lập trình Arduino IDE và ngôn ngữ Sketch 4

6.4.1 Chu trình công cụ và chương trình 4

6.4.2 Hai hàm `setup()` và `loop()` 4

6.4.3 Lệnh và khái niệm quan trọng 4

6.4.4 Ví dụ: đọc analog và gửi Serial 5

6.4.5 Serial UART và COM 5

6.5 ESP8266 và các module phổ biến 5

6.5.1 ESP-01, ESP-07 và NodeMCU 6

6.5.2 Firmware, SDK và quy trình IDE 6

6.6 ESP32: một họ SoC, không phải một chip duy nhất 6

6.6.1 Lợi thế và giới hạn thực dụng 6

6.6.2 So sánh các lựa chọn MCU căn bản 7

6.7 Intel Edison: ví dụ nền tảng lai có tính lịch sử 7

6.8 Edge Computing trong kiến trúc IoT 7

6.8.1 So sánh các vị trí xử lý 8

6.9 Raspberry Pi: máy tính đơn bo cho gateway và Edge 8

6.9.1 Các thành phần và GPIO 8

6.9.2 Dựng gateway cơ bản với Raspberry Pi 9

6.9.3 Arduino và Raspberry Pi: không có nền tảng nào tốt nhất cho mọi việc . 9

6.10 Thiết kế nút IoT: bài toán tổng hợp nhiều nền tảng 9

6.10.1 Bảng quyết định chọn phần cứng 9

6.10.2 Công thức sơ bộ về lưu trữ và truyền thông 10

6.10.3 Ví dụ hệ IoT đầy đủ: giám sát phòng học 10

6.11 Câu hỏi tự luyện và đáp án ngắn 10

6.12 Bảng thuật ngữ ôn thi 11

CHƯƠNG 6

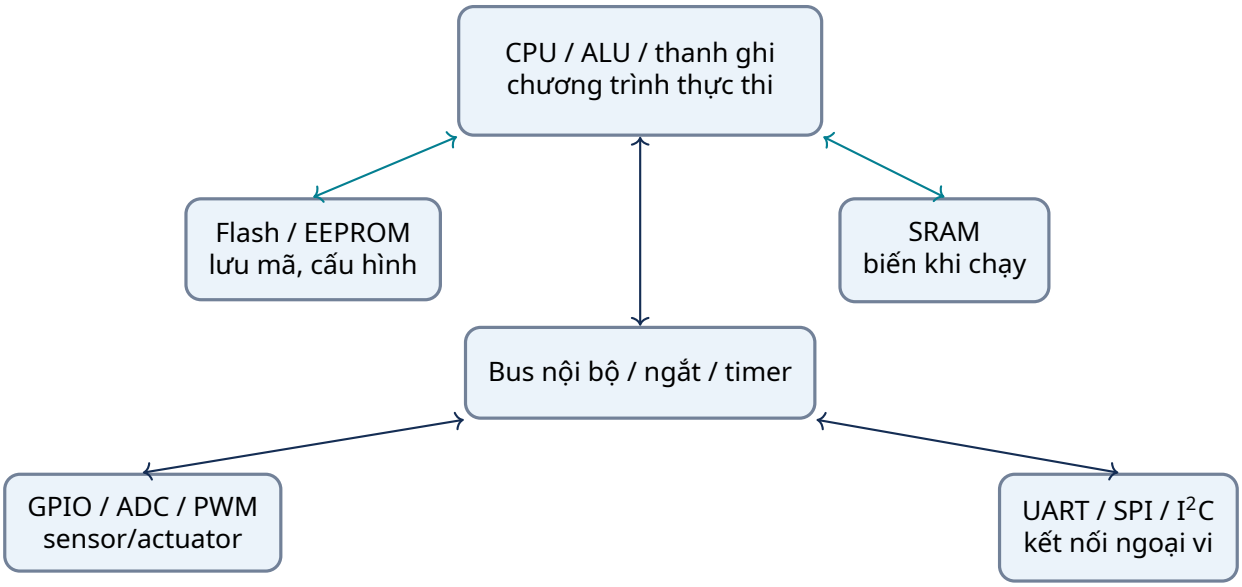
Vi điều khiển và nền tảng phần cứng IoT

6.1 Kiến trúc hệ thống nhúng và mục tiêu học tập

Sau chương này, người học cần hiểu sự khác nhau giữa chip vi điều khiển, bo phát triển và máy tính đơn bo; biết cấu trúc CPU–Flash–RAM–I/O; mô tả Arduino Uno, Leonardo, Mega, Nano, ATmega328P, ESP8266, ESP32, Intel Edison, Raspberry Pi; hiểu thao tác lập trình Arduino IDE, nối cảm biến, dùng UART và lựa chọn xử lý ở Edge hay Cloud.

Vi điều khiển (*microcontroller/MCU*) là vi mạch tích hợp bộ xử lý, bộ nhớ và ngoại vi vào một hệ lập trình được, tối ưu cho các tác vụ nhúng. Một **bo phát triển** (*development board*) đưa chip ra các chân tiện dụng và có thêm mạch nguồn, USB, clock, bộ nạp. Một **máy tính đơn bo** (*single-board computer/SBC*) như Raspberry Pi thường có SoC mạnh hơn, RAM ngoài, lưu trữ và khả năng chạy hệ điều hành đa nhiệm. Do đó **Arduino không đồng nghĩa với chip ATmega**, và **Raspberry Pi không nên được coi là MCU theo nghĩa hẹp**.

Theo Chapter 2: IoT Hardware, slide PDF 18, 22–26, 73–78.



Hình 6.1: Kiến trúc MCU mức khái niệm (sơ đồ mới, không phải sơ đồ silicon cụ thể).

6.2 ATmega328P: nền tảng AVR 8 bit

Slide trình bày ATmega328P là MCU AVR 8-bit hiệu năng tốt, công suất thấp, bộ nhớ Flash 32 KB, SRAM 2 KB, EEPROM 1 KB, tối đa 23 đường I/O lập trình được trên chip, dải điện áp hoạt động tùy tần số/phiên bản 1.8–5.5 V, vùng nhiệt độ công bố trong slide -40 đến $85\,^{\circ}$C. Các giá trị dòng tiết kiệm năng lượng đưa ra trong slide (active 0.2 mA, power-down $0.1\,\mu$A, power-save $0.75\,\mu$A) *chỉ là số liệu dưới điều kiện kiểm thử riêng*, không thể mặc nhiên dùng cho bo Arduino Uno khi có LED, bộ ổn áp và USB-UART.

Theo Chapter 2: IoT Hardware, slide PDF 19–21.

6.2.1 Flash, SRAM, EEPROM: ba vai trò khác nhau

Bảng 6.1: Bộ nhớ của ATmega328P theo bài giảng

Loại	Dung lượng	Chức năng
Flash	32 KB	Chứa chương trình và dữ liệu tĩnh, giữ khi mất nguồn
SRAM	2 KB	Biến lúc chạy, stack/heap; mất khi tắt nguồn
EEPROM	1 KB	Lưu cấu hình nhỏ khi mất nguồn; có giới hạn số chu kỳ ghi

Nếu khai báo quá nhiều biến động (mảng lớn, chuỗi, bộ đệm), hệ thống có thể lỗi do SRAM cạn dù Flash còn chỗ. Với dữ liệu ghi lặp nhiều, phải quan tâm mài mòn EEPROM và chiến lược ghi tối thiểu.

6.2.2 Chân nguồn, I/O và nạp chương trình

ATmega328P ở gói PDIP 28 chân thường có VCC, AVCC, GND, AREF, RESET, XTAL1/XTAL2 và các nhóm PORTB/C/D. Một chân vật lý có thể kiêm nhiều chức năng (GPIO, ADC, UART, SPI, PWM, ngắt), do đó *số chân package* không phải số GPIO thực dùng độc lập. Trong slide, giao tiếp nạp chương trình minh họa các tín hiệu RESET, SCK, MOSI, MISO, VCC, GND theo chuẩn ISP. Bootloader Arduino giúp nạp sketch qua USB-to-serial trên bo, trong khi ISP cho phép lập trình trực tiếp khi cần.

Theo Chapter 2: IoT Hardware, slide PDF 19, 21, 25, 27.

6.3 Arduino: hệ sinh thái phần cứng/phần mềm mở

Arduino là nền tảng điện tử mã nguồn mở gắn với bo và công cụ lập trình dễ tiếp cận, hỗ trợ nhiều hệ điều hành. Slide nhấn mạnh chi phí thấp, hỗ trợ đa nền tảng, môi trường lập trình đơn giản, phần mềm và phần cứng mở. Nội dung lịch sử trong slide nhắc dự án xuất phát từ môi trường giáo dục thiết kế tương tác tại Ivrea (Ý), phát triển thành hệ sinh thái bo, thư viện và shield rộng lớn.

Theo Chapter 2: IoT Hardware, slide PDF 22–24.

6.3.1 Các bo Arduino trong bài giảng

Slide còn nhắc EEPROM 1 KB trên Uno/Nano/Leonardo, 4 KB trên Mega; dòng chân I/O và dải nguồn phải xét theo MCU, điện áp cấp *cho chip* khác với điện áp cho chân VIN hoặc jack bo mạch. Một số số lượng digital pin trong slide bao gồm chân đa chức năng, do vậy cần xem

Bảng 6.2: Các bo điển hình theo slide; số liệu áp dụng cho phiên bản cổ điển được nêu

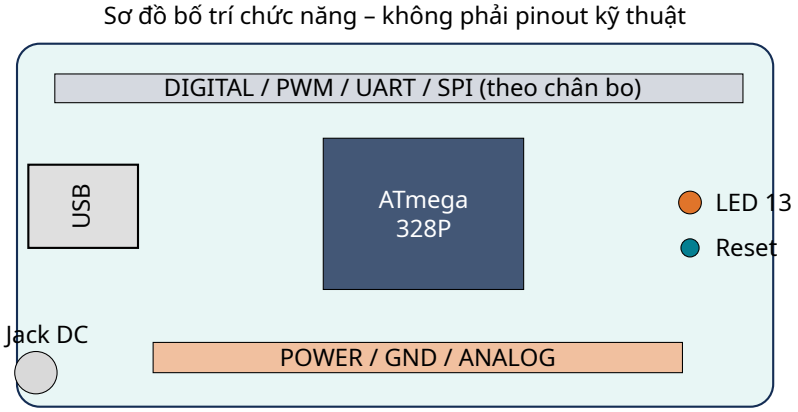
Bo	MCU	Flash / SRAM	Đầu vào/ra và ghi chú
Uno Rev3 (cổ điển)	ATmega328P, 8-bit	32 KB / 2 KB	14 chân digital, 6 analog; USB loại B; bootloader
Leonardo	ATmega32U4, 8-bit	32 KB / 2.5 KB	Tài liệu slide nêu 23 digital, 12 analog; USB tích hợp MCU
Mega 2560	ATmega2560, 8-bit	256 KB / 8 KB	54 digital, 16 analog; phù hợp khi cần nhiều chân
Nano (ATmega328)	ATmega328, 8-bit	32 KB / 2 KB	Thiết kế nhỏ; slide nêu 8 ngõ analog, 22 digital tổng hợp

sơ đồ chân của *phiên bản bo* cụ thể. Arduino Uno mới (ví dụ R4) không dùng ATmega328P; không thể suy thông số từ bảng này cho mọi sản phẩm tên "Uno".

Theo Chapter 2: IoT Hardware, slide PDF 25–30.

6.3.2 Phân tích một bo Uno cổ điển

Sơ đồ trang 26 slide chỉ ra các khu vực: hàng chân Digital, LED chân 13, LED báo nguồn, ATmega chính, hàng Analog, chân GND/5V, jack nguồn, đèn TX/RX, cổng USB và nút Reset. Chân Arduino D0/D1 thường gắn với UART RX/TX, nên dùng đồng thời với module UART và Serial Monitor có thể gây tranh chấp.



Hình 6.2: Sơ đồ khối bố trí thành phần tiêu biểu của Arduino Uno đời ATmega328P.

6.3.3 Shield, breadboard, kit và các nền tảng thay thế

Shield (*bo mở rộng*) cắm vào một số bo Arduino qua hàng chân tiêu chuẩn để bổ sung Ethernet, relay, màn hình, driver mô-tơ hoặc kết nối RF. Cần kiểm tra xung đột chân và tương thích mức logic. **Breadboard** (*bảng cắm thử không hàn*) thích hợp thử nghiệm nguyên mẫu, nhưng không lý tưởng cho dòng lớn, điện áp cao, tần số RF và rung động. Arduino kit thường gom các linh kiện, dây, sensor, servo và tài liệu thực hành.

Các phương án được slide giới thiệu gồm LaunchPad MSP430, Particle Photon, NodeMCU ESP8266, SparkFun Thing Plus, Feather HUZZAH, PocketBeagle, Seeeduino Nano, RedBoard Artemis và STM32F3 Discovery. Mỗi nền tảng có hệ sinh thái, CPU, mức logic và công cụ nạp khác nhau; được liệt kê để so sánh *khả năng lựa chọn*, không phải toàn bộ tương thích chân và mã Arduino.

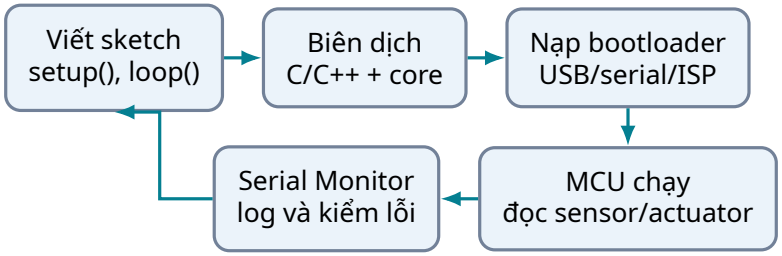
Theo Chapter 2: IoT Hardware, slide PDF 31–37.

6.4 Lập trình Arduino IDE và ngôn ngữ Sketch

6.4.1 Chu trình công cụ và chương trình

Slide gọi chương trình tải lên Arduino là **sketch** (*bản phác chương trình*); ngôn ngữ thực tế dựa trên C/C++ và các hàm của Arduino core. IDE cho phép lựa chọn board, cổng serial, quản lý thư viện, biên dịch, nạp và quan sát Serial Monitor. Quá trình cài board bên thứ ba (như ESP8266) thường đi qua Boards Manager; thư viện cài qua Library Manager hoặc các hình thức khác tùy dự án.

Theo Chapter 2: IoT Hardware, slide PDF 38–41, 51–52.



Hình 6.3: Chu trình thiết kế – nạp – kiểm thử firmware Arduino.

6.4.2 Hai hàm `setup()` và `loop()`

`setup()` chạy một lần sau khi khởi động hoặc reset để cấu hình chân, serial, thư viện và trạng thái ban đầu. `loop()` chạy lặp lại; muốn hệ thống phản ứng nhanh với nhiều công việc, nên ưu tiên lập lịch theo `millis()` thay vì nhiều lệnh `delay()` dài. Khi có lỗi hoặc yêu cầu dừng vĩnh viễn cần xử lý có chủ đích, tránh khối vòng lặp vô hạn vô tình.

Theo Chapter 2: IoT Hardware, slide PDF 41–51.

Listing 6.1: Cấu trúc sketch tối thiểu

```
const int LED_PIN = LED_BUILTIN;
void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(500);
  digitalWrite(LED_PIN, LOW);
  delay(500);
}
```

6.4.3 Lệnh và khái niệm quan trọng

Bảng 6.3: Tóm tắt cú pháp được giới thiệu trong slide 40–50

Lệnh / cấu trúc	Ý nghĩa và chú ý
`//`, `/*...*/`	Chú thích một dòng và nhiều dòng; trình biên dịch bỏ qua nội dung chú thích.
`#define`, `#include`	Định nghĩa macro và thêm phần khai báo thư viện/header; khác với khai báo biến.

Lệnh / cấu trúc	Ý nghĩa và chú ý
`if/else`	Rẽ nhánh nếu điều kiện đúng/sai; điều kiện là biểu thức logic.
`for`, `while`	Lặp có đếm hoặc lặp khi điều kiện còn đúng; đề phòng vòng lặp vô hạn.
`pinMode(pin,mode)`	Đặt chế độ chân (`INPUT`, `OUTPUT`, `INPUT_PULLUP` khi hỗ trợ).
`digitalRead(pin)`	Đọc trạng thái logic chân số (HIGH/LOW).
`digitalWrite(pin,value)`	Ghi mức HIGH/LOW lên GPIO output.
`analogRead(pin)`	Đọc ADC trả mã số; độ rộng/mức tham chiếu tùy bo.
`analogWrite(pin,value)`	Trên Uno cổ điển tạo PWM ở các chân hỗ trợ; độ rộng giá trị mặc định thường 0–255.
`delay(ms)`	Dừng tác vụ một khoảng thời gian (không nên dùng tràn lan trong tác vụ thời gian thực).
`abs()`, `max()`, `min()`, `pow()`	Các hàm toán thường dùng; để ý kiểu dữ liệu và tràn số.
`Serial.begin(baud)`	Khởi tạo tốc độ serial; tốc độ phía giám sát phải khớp.
`Serial.print/println()`	Gửi chuỗi/số lên terminal để theo dõi/debug.

6.4.4 Ví dụ: đọc analog và gửi Serial

Listing 6.2: Đọc ADC của Arduino Uno ATmega328P

```
const int ADC_PIN = A0;
void setup() {
  Serial.begin(9600);
}
void loop() {
  int raw = analogRead(ADC_PIN); // Uno: nominal 0..1023
  float voltage = raw * (5.0f / 1023.0f);
  Serial.print("ADC="); Serial.print(raw);
  Serial.print(" V="); Serial.println(voltage, 3);
  delay(250);
}
```

Công thức trên *giả định tham chiếu ADC đúng 5.0 V* và nguồn nằm trong mức an toàn, chỉ dùng để minh họa. Cần đo/tham chiếu Vref và xử lý sai số nếu đòi hỏi phép đo chính xác.

6.4.5 Serial UART và COM

Serial port truyền dữ liệu nối tiếp. Trong máy tính, “COM” có thể là cổng vật lý hoặc cổng ảo sinh bởi USB-to-serial, Bluetooth serial hoặc driver khác. Slide ghi chân 0 của Uno là RX, chân 1 là TX. UART TTL là mức logic của chip, khác *điện áp chuẩn RS-232*; không được nối trực tiếp phần cứng RS-232 truyền thống vào GPIO TTL nếu không có transceiver thích hợp. Mức baud, định dạng khung, GND và dây TX/RX phải được kiểm tra.

Theo Chapter 2: IoT Hardware, slide PDF 52–53.

6.5 ESP8266 và các module phổ biến

ESP8266EX (*Wi-Fi SoC*) của Espressif là vi mạch giá thấp tích hợp Wi-Fi và stack TCP/IP, có thể chạy ứng dụng độc lập hoặc đóng vai trò giao tiếp cho vi điều khiển khác. Slide đề cập

TCP, UDP, HTTP, mDNS, DNS, OTA, Flash/SD, SPI, I^2C và hệ Arduino core; việc hỗ trợ từng tính năng phụ thuộc firmware, core, thư viện và chân được đưa ra module. ESP8266 chủ yếu hoạt động ở mức logic 3.3 V: không cấp trực tiếp tín hiệu GPIO 5 V nếu không xác minh và chuyển mức.

Theo Chapter 2: IoT Hardware, slide PDF 54–61.

6.5.1 ESP-01, ESP-07 và NodeMCU

Bảng 6.4: Những dạng phần cứng ESP8266 xuất hiện trong slide

Module / bo	Điểm cần nhớ
ESP-01	Module nhỏ (slide nêu khoảng 24.75×14.5 mm), anten PCB, chân hạn chế, nguồn 3.3 V; firmware AT phổ biến cho điều khiển qua UART.
ESP-07	Module 3.3 V; slide nêu anten gốm và đầu nối U.FL, Wi-Fi 2.4 GHz b/g/n, TCP/UDP; chú ý chân pitch nhỏ.
NodeMCU	Bo phát triển dùng module ESP8266, có USB-UART, bộ ổn áp và đưa chân ra hàng header; sơ đồ nhãn D0/D1... không đồng nhất số GPIO silicon.

6.5.2 Firmware, SDK và quy trình IDE

Slide đề cập Espressif SDK NonOS, RTOS SDK, ESP-Open-SDK và các hệ Arduino/ESP Easy/ESPHome. Đây là các lựa chọn có lịch sử phát triển riêng, không phải cùng một thư viện. Đối với ví dụ lịch sử trên slide, cài ESP8266 Arduino core bằng cách thêm Boards Manager URL rồi cài package ESP8266 và chọn Generic ESP8266 Module; *URL và phiên bản package có thể thay đổi*, phải xem tài liệu chính thức hiện hành.

Theo Chapter 2: IoT Hardware, slide PDF 58–61.

6.6 ESP32: một họ SoC, không phải một chip duy nhất

ESP32 được Espressif phát triển với nhiều bộ xử lý, ngoại vi và khả năng vô tuyến tùy *dòng chip*. Slide mô tả ESP32 đời đầu có hai lõi, khoảng 160–240 MHz, hỗ trợ Wi-Fi/Bluetooth, nhiều GPIO và công năng kết nối cao hơn ESP8266, chi phí cao hơn. Tuy nhiên, có ESP32-S/C/H và các biến thể *không có cùng số lõi, Bluetooth, Wi-Fi, chân ADC hoặc giao diện USB*; không được áp dụng nguyên bảng thông số của ESP32 đời đầu cho toàn bộ họ ESP32.

Theo Chapter 2: IoT Hardware, slide PDF 62–65.

6.6.1 Lợi thế và giới hạn thực dụng

- **Tích hợp:** MCU và phần giao tiếp vô tuyến trên một chip/module giúp tiết kiệm diện tích và thời gian phát triển.
- **Ngoại vi:** GPIO, SPI, UART, I^2C, PWM, ADC, timer, interrupt; số lượng và ma trận ánh xạ chân phụ thuộc mã chip.
- **Nguồn:** phát Wi-Fi/BLE có dòng đỉnh cao; cần tụ decoupling và bộ ổn áp có dự trữ dòng. Giảm công suất bằng sleep hoặc giảm tần suất truyền khi tác vụ cho phép.
- **Phần mềm:** Arduino core thuận tiện học tập; ESP-IDF thích hợp khai thác phần mềm hệ thống, RTOS và ngoại vi chuyên sâu.

- **Tương thích:** cần kiểm tra chân bootstrap, chân dành riêng Flash, chân input-only (với một số chip), miền ADC, mức logic, loại giao thức Bluetooth cụ thể.

Ghi nhớ

Nhận định trên slide "ESP32 ra đời do ESP8266 thiếu bảo mật": nội dung bài giảng có đề cập như vậy, nhưng không nên xem đây là lời giải thích lịch sử đầy đủ. ESP32 còn mở rộng năng lực xử lý, ngoại vi và tính năng nền tảng. Các cơ chế bảo mật thực tế phụ thuộc dòng chip, boot configuration, firmware và cách triển khai sản phẩm.

6.6.2 So sánh các lựa chọn MCU căn bản

Bảng 6.5: So sánh định tính khi chọn nền tảng; không thay datasheet

Đặc điểm	ATmega328P/Uno	ESP8266	ESP32 (họ chip)
Kết nối mặc định	Không có IP/Wi-Fi trên MCU	Wi-Fi 2.4 GHz	Tùy dòng: Wi-Fi và/hoặc Bluetooth; không đồng nhất
Hệ logic	Bo Uno cổ điển 5 V	Chủ yếu 3.3 V	Chủ yếu 3.3 V
Năng lực xử lý	8-bit, RAM rất nhỏ	MCU 32-bit	Tùy dòng, nhiều ngoại vi
Lập trình	Arduino IDE, AVR-GCC	Arduino core/SDK	Arduino core/ESP-IDF
Phù hợp	Điều khiển cơ bản, học GPIO	Nút IoT Wi-Fi giá thấp	Nút IoT đa cảm biến, kết nối, Edge nhỏ

6.7 Intel Edison: ví dụ nền tảng lai có tính lịch sử

Theo slide, module Intel Edison có kích thước xấp xỉ $35.5 \times 25 \times 3.9$ mm, SoC Atom "Tangier" sản xuất tiến trình 22 nm, hai lõi Silvermont 500 MHz, lõi Quark 100 MHz, RAM 1 GB, eMMC 4 GB, hỗ trợ Wi-Fi, Bluetooth 4, USB và đầu nối 70 chân. Các bo mở rộng được giới thiệu gồm *breakout board* mức I/O 1.8 V với USB OTG, cổng cấp nguồn, sạc và cầu UART; *Arduino expansion board* đưa thêm chân kiểu Arduino (slide liệt kê 20 digital, 6 analog, UART, I^2C, SPI/ICSP, microSD).

Theo Chapter 2: IoT Hardware, slide PDF 66–68.

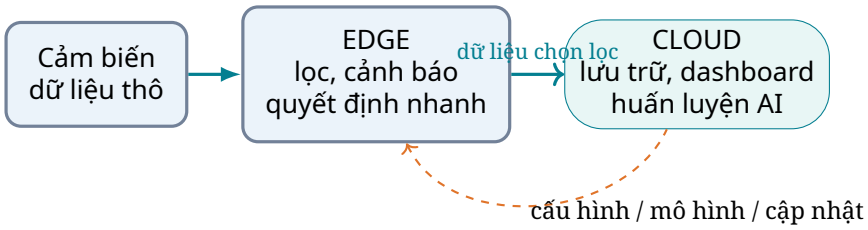
Ghi nhớ

Góc nhìn lịch sử: Intel Edison là nền tảng đã ngừng sản xuất/hỗ trợ. Chương này giữ để đọc hiểu slide và so sánh mô hình SoC + breakout + bo mở rộng, *không khuyến nghị* chọn làm nền tảng cho thiết kế sản phẩm mới. I/O 1.8 V cần đặc biệt chú ý khi nối module ngoài.

6.8 Edge Computing trong kiến trúc IoT

Điện toán biên (*edge computing*) chuyển một phần thu nhận, lọc, phân tích hoặc quyết định đến thiết bị gần nơi sinh dữ liệu hơn thay vì gửi toàn bộ dữ liệu thô lên đám mây. Bài giảng minh họa các tình huống *nhạy trễ*, *nhạy riêng tư*, bị hạn chế băng thông hoặc kết nối chập chờn; các thiết bị edge tiêu biểu có thể là MCU đủ mạnh, gateway Linux hoặc SBC.

Theo Chapter 2: IoT Hardware, slide PDF 69–72.



Hình 6.4: Sự phân công công việc điển hình giữa Edge và Cloud.

6.8.1 So sánh các vị trí xử lý

Bảng 6.6: Edge và Cloud: tiêu chí thiết kế

Tiêu chí	Xử lý Edge	Xử lý Cloud
Độ trễ	Có thể thấp, không nhất thiết phụ thuộc Internet	Có thêm trễ mạng và dịch vụ
Băng thông	Có thể lọc/gộp dữ liệu trước khi gửi	Có khả năng nhận tập dữ liệu lớn
Kết nối kém	Có thể tiếp tục một số chức năng	Khó dùng khi mất Internet
Tài nguyên	Giới hạn bởi công suất tính toán, pin, RAM	Co giãn dễ hơn, nhưng có phí/độ trễ
Riêng tư	Giảm dữ liệu thô rời thiết bị nếu thiết kế đúng	Cần chính sách truy cập và lưu trữ phù hợp

6.9 Raspberry Pi: máy tính đơn bo cho gateway và Edge

Raspberry Pi là họ máy tính đơn bo kích thước nhỏ, xuất hiện trong bài giảng như lựa chọn chạy hệ điều hành, kết nối mạng, làm gateway, hiển thị và phân tích dữ liệu. Khác với Uno, Raspberry Pi có thể chạy Linux và nhiều chương trình đồng thời, song khởi động chậm hơn, cần nguồn ổn định và bảo vệ lưu trữ. Slide đưa ví dụ Raspberry Pi 4 và những thành phần của kit: bo Pi, thẻ microSD, nguồn, bàn phím/chuột USB, màn hình HDMI, cáp HDMI, vỏ bảo vệ.

Theo Chapter 2: IoT Hardware, slide PDF 72–77.

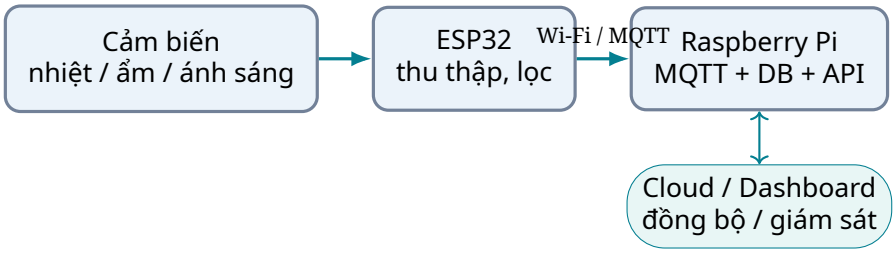
6.9.1 Các thành phần và GPIO

Raspberry Pi 4 minh họa trong slide có các khu vực: SoC, RAM, cổng USB, Ethernet, HDMI dạng micro, USB-C cấp nguồn, đầu camera/display và hàng chân mở rộng 40-pin. **GPIO của Raspberry Pi hoạt động theo logic 3.3 V**, không được đưa trực tiếp điện áp 5 V vào GPIO nếu không có mạch giao tiếp phù hợp. Số chân header vật lý, số GPIO và chức năng alternate không đồng nhất.

Cần phân biệt: USB-C trên Raspberry Pi 4 chủ yếu dùng cấp nguồn; điều đó không đồng nghĩa mọi Pi có thể lập trình/nạp trực tiếp như ESP32 qua cùng cổng. Với Pi chạy Linux, lập trình thường thông qua SSH, terminal, editor hoặc copy file lên hệ thống; môi trường và cách boot khác MCU.

6.9.2 Dựng gateway cơ bản với Raspberry Pi

Một kiến trúc ví dụ: cảm biến nối với ESP32, ESP32 truyền dữ liệu qua Wi-Fi/MQTT đến broker trên Raspberry Pi, Pi lưu cơ sở dữ liệu/đẩy lên dashboard/cloud. Nếu mạng Internet bên ngoài mất, dữ liệu có thể vẫn được lưu ở Pi trong LAN; khi có mạng lại, tiến hành đồng bộ theo chính sách ứng dụng.



Hình 6.5: Ví dụ nguyên lý mạng IoT tại nhà dùng ESP32 và Raspberry Pi.

6.9.3 Arduino và Raspberry Pi: không có nền tảng nào tốt nhất cho mọi việc

Bảng 6.7: Bảng kết luận đối chiếu nội dung cuối slide

Thuộc tính	Arduino Uno (ATmega328P)	Raspberry Pi 4 (SBC)
Bộ xử lý	MCU 8-bit điều khiển đơn giản	SoC mạnh chạy Linux
Hệ điều hành	Sketch chạy trên MCU, không có Linux	Hệ điều hành và đa tiến trình
Thời gian khởi động	Thường nhanh và đơn giản	Cần boot OS từ thẻ/lưu trữ
Tính xác định	Thuận lợi cho I/O định thời đơn giản	Linux thông thường không bảo đảm hard real-time
Mạng	Cần shield/module nếu muốn Wi-Fi	Có giao tiếp mạng tích hợp ở một số model
Ứng dụng	Cảm biến, đóng ngắt, điều khiển thời gian đơn giản	Gateway, database, dashboard, xử lý hình ảnh
Chi phí, điện năng	Thường thấp hơn ở lớp nút đơn giản	Thường cao hơn về điện năng/tài nguyên

Theo Chapter 2: IoT Hardware, slide PDF 78.

6.10 Thiết kế nút IoT: bài toán tổng hợp nhiều nền tảng

6.10.1 Bảng quyết định chọn phần cứng

Bảng 6.8: Checklist thiết kế từ yêu cầu tới linh kiện

Câu hỏi	Quyết định kỹ thuật tương ứng
Dữ liệu cần đo là gì?	Cảm biến, số trục, khoảng đo, accuracy, giao tiếp và tốc độ lấy mẫu
Tác động tới môi trường?	LED/servo/motor, driver, điện áp, dòng đỉnh và mạch bảo vệ

Câu hỏi	Quyết định kỹ thuật tương ứng
Làm việc khi mất mạng?	Đặt xử lý thiết yếu trên MCU/Edge; đệm dữ liệu và xác thực gửi lại
Tốc độ/khối lượng xử lý?	MCU 8-bit, ESP32, SBC Linux, bộ nhớ cần thiết
Cần Wi-Fi/BLE/RF?	SoC tích hợp hay module ngoài, băng thông, vùng phủ, anten
Nguồn pin hay nguồn cố định?	Dòng active/sleep/peak, nguồn ổn áp và sạc, thời lượng mục tiêu
Dung lượng dữ liệu?	RAM, Flash, EEPROM, microSD, độ bền ghi và khả năng phục hồi
Bảo mật?	Danh tính thiết bị, xác thực broker/API, TLS khả thi, firmware OTA an toàn
Độ ổn định vận hành?	Watchdog, reset, brownout, lọc nhiễu, xác minh wiring và logging
Bảo trì về sau?	Datasheet, sơ đồ chân, BOM, README, phiên bản firmware, test plan

6.10.2 Công thức sơ bộ về lưu trữ và truyền thông

Nếu một mẫu có B byte và tốc độ f_s mẫu/giây, trong thời gian T giây thì dữ liệu thuần tối thiểu:

$$\boxed{M = B\, f_s\, T} \quad \text{byte}, \tag{6.1}$$

chưa tính timestamp, JSON/CSV, header mạng, kiểm tra lỗi và hệ số dự phòng. Ví dụ mỗi mẫu 24 byte, 10 Hz trong 1 giờ: $24 \cdot 10 \cdot 3600 = 864{,}000$ byte (khoảng 0.864 MB hệ thập phân). Khi ghi microSD/Flash phải tính cả kích thước block và tuổi thọ ghi.

6.10.3 Ví dụ hệ IoT đầy đủ: giám sát phòng học

1. **Nhu cầu:** giám sát nhiệt độ/độ ẩm, độ sáng và thông báo nếu vượt ngưỡng; đồng thời quản lý quạt thông gió.
2. **Cảm biến:** chọn module nhiệt–ẩm số và cảm biến ánh sáng, đánh giá dải đo, sai số và điều kiện lắp.
3. **Bộ xử lý:** chọn ESP32 nếu cần Wi-Fi trực tiếp; chọn Uno kèm module Wi-Fi nếu cần học các khối tách rời; chọn Raspberry Pi làm gateway/dashboard.
4. **Actuator:** relay/MOSFET phù hợp điện áp, dòng tải và chuẩn an toàn; thiết lập trạng thái fail-safe nếu MCU lỗi.
5. **Phần mềm:** ước lượng chu kỳ lấy mẫu, chuẩn hóa dữ liệu, timestamp, MQTT hoặc HTTP, cấu hình ngưỡng và xử lý mất kết nối.
6. **Kiểm thử:** chạy đo trong điều kiện bình thường, biên cao/thấp, rút mạng, cấp nguồn lại, thử dòng đỉnh của tải và log lỗi.

6.11 Câu hỏi tự luyện và đáp án ngắn

1. **MCU khác SBC ở đâu?** – MCU tích hợp CPU, RAM, Flash và ngoại vi nhúng; SBC là hệ máy tính hoàn chỉnh thường chạy OS và có nhiều tài nguyên hơn.

2. **ATmega328P có mấy loại bộ nhớ chính theo slide?** – Flash 32 KB, SRAM 2 KB, EEPROM 1 KB.
3. **Arduino Uno cổ điển có bao nhiêu ngõ digital và analog?** – 14 digital và 6 analog được đưa ra chân trên bo, không đồng nghĩa chip chỉ có từng đó chức năng.
4. **Vai trò `setup` và `loop`?** – Cấu hình một lần; vòng xử lý lặp liên tục.
5. **Serial port chân nào trên Uno?** – D0/RX và D1/TX; GND và mức điện áp phải tương thích.
6. **ESP-01 khác NodeMCU?** – ESP-01 là module nhỏ với ít chân; NodeMCU là bo phát triển tích hợp USB-UART/nguồn, tiện lập trình.
7. **ESP32 có luôn dual-core và Bluetooth Classic?** – Không; đặc tính tùy series/model.
8. **Tại sao Intel Edison xuất hiện trong bài?** – Ví dụ lịch sử về module SoC + breakout/Arduino board; không dùng làm lựa chọn hiện đại mặc định.
9. **Khi nào cần Edge?** – Khi ưu tiên trễ thấp, giảm băng thông, bảo vệ riêng tư và duy trì tác vụ tại chỗ khi mất mạng.
10. **Raspberry Pi khác Arduino về môi trường chạy?** – Pi thường chạy Linux; Uno chạy sketch nhúng, không có hệ điều hành đa nhiệm tổng quát như Linux.

6.12 Bảng thuật ngữ ôn thi

Thuật ngữ	Diễn giải vắn tắt
MCU	Microcontroller Unit – vi điều khiển đơn chip tích hợp CPU, bộ nhớ, ngoại vi.
SoC	System-on-Chip – hệ thống tích hợp trên chip, thường có CPU, ngoại vi và khối chuyên dụng.
SBC	Single-board Computer – máy tính trên một bo, thường chạy OS.
GPIO	General-Purpose Input/Output – chân nhập/xuất số đa dụng.
ADC	Analog-to-Digital Converter – bộ chuyển tương tự sang số.
PWM	Pulse-Width Modulation – điều chế độ rộng xung.
UART	Universal Asynchronous Receiver/Transmitter – nối tiếp bất đồng bộ.
SPI	Serial Peripheral Interface – giao tiếp nối tiếp đồng bộ, thường có CS.
I^2C	Bus SDA/SCL hai dây, nhiều thiết bị theo địa chỉ.
Bootloader	Phần mềm khởi động hỗ trợ nạp firmware qua giao diện quy định.
OTA	Over-the-Air – cập nhật firmware qua mạng vô tuyến.
Shield	Bo cắm mở rộng cho một nền tảng (ví dụ Arduino).
Breakout board	Bo đưa các chân/tính năng của module/chip ra chân kết nối dễ dùng.
Edge Computing	Xử lý ở gần nơi sinh dữ liệu thay vì chuyển hết về Cloud.
MQTT	Giao thức publish/subscribe nhẹ phổ biến trong IoT, sử dụng broker.

Ghi nhớ

Tóm tắt cần nhớ: Phân biệt *MCU – module – board – SBC*. ATmega328P/Arduino thích hợp nền tảng học điều khiển; ESP8266/ESP32 bổ sung kết nối không dây và năng lực xử lý; Intel Edison là tư liệu lịch sử; Raspberry Pi hợp lý cho gateway và ứng dụng Linux. Edge giúp giảm độ trễ và phụ thuộc Cloud. Luôn đối chiếu sơ đồ chân, điện áp logic và nguồn trước khi lắp mạch.